

1) CPU Scheduling in Real-time Ticket

P1 → Payment Gateway (priority ↑)

P2 → Medium priority.

P3 → Low priority

BT = 2 to 20 ms.

A customers complain that ticket are delayed.

Justification:

- * high priority task executes
- * Improves response time

Sample execution order

High	P1	payment gateway
Low	P3	Ticket confirmation
Medium	P2	Seat availability.

Execution Order

P1 → P2 - P3

Drawback

May cause starvation, wait for long time if ^{high} priority tasks keep arriving.

2) Deadlock Identification.

Yes, the system is in deadlock because there is a circular wait.

* T1 holds A and waits for B.

* T2 holds B and waits for A.

* T3 holds C and waits for A.

Since every transaction waits for another, none of them can continue.

(*) Techniques used:

used deadlock prevention by allocating resource in a fixed order.

(*) Locking order

$A \rightarrow B \rightarrow C$

3) Mobile OS App switching Log.

(*) Why CPU is idle:

* most applications are waiting for I/O operations such as disk or network access.

(*) Time Quantum Analysis:

The current quantum of 20ms is too large.

②

It delays other applications from getting CPU time and reduce response.

(*) Recommendation:

Use Round Robin with 5-10 ms quantum or multi-level feedback queue (MLFQ). These methods improve response time & CPU scheduling.

(*) Performance Improvement.

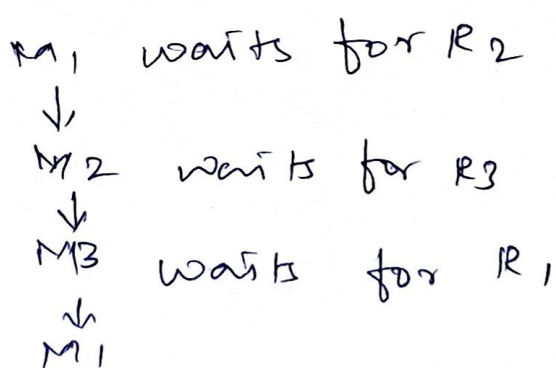
A small quantum allows application to get CPU time more frequently resulting in smoother app & better throughput.

4) Deadlock Risk During os update:

(*) Can deadlock occur?

Yes, given resources allocation lead to deadlock.

(*) Circular main condition?



1*) Banker Algorithms :

Allocate resources only if the system remains in a safe state. If granting a resource request makes the system.

1*) Module Priority :

- * M₁ - Highest
- * M₂ - Medium
- * M₃ - medium
- * M₄ - Lowest.

5) Cloud Server CPU Utilization:

Given:

* I/O wait probability $(P) = 0.8$

* no. of. UMS $(n) = 7$

Formula:

$$\text{CPU Utilization} = 1 - P^n$$

Calculation:

$$= 1 - (0.8)^7$$

$$= 1 - 0.2097152$$

$$= 0.7902848$$

$$= 79.03\%$$

(*) Interpretation:

the server is well utilized because the CPU is busy about 79% of the time.

(*) Effects of VMS:

- * Increasing VMS improve CPU utilization throughput.

- * Too many VMS increase context switch overhead.

- * Decreasing VMS reduces CPU utilization & increase idle time.

(*) OS level strategies:

- * increase the degree of multiprogram.

- * Use efficient CPU scheduling such as MLFQ or optimized Round Robin.

- * Reduce unnecessary I/O waiting to improve CPU utilization.